

CS6888 Software Analysis and its Applications

HW3

Sravanth Chowdary Potluri
und5uv@virginia.edu
University Of Virginia

I. TCAS1

A. Analysis of Suspicious Statements

(a) *Faulty Statement Identification:* The actual fault in TCAS1 is in the `Inhibit_Biased_Climb()` function. The faulty line is:

```
return (Climb_Inhibit ? Up_Separation  
+ NOZCROSS : Up_Separation - NOZCROSS);
```

located at Line 61. However, this line is ranked only 8th with a suspiciousness score of 1.36. In contrast, the top-ranked statement (Line 134: `alt_sep = DOWNWARD_RA;`) has a score of 5.52. This discrepancy occurs because the faulty line is executed in both failing and passing tests, thus diluting its suspiciousness. Meanwhile, Line 134 is executed almost exclusively in failing test cases, making its correlation with failures stronger and resulting in a much higher suspiciousness score.

(b) *Impact on Debugging:* A developer unaware of the fault's location might first focus on the top-ranked statement at Line 134. Upon investigating why `alt_sep` is set to `DOWNWARD_RA`, the developer would trace the logic and eventually inspect the decision conditions, including the function `Inhibit_Biased_Climb()`. Although the faulty statement (Line 61) is not the highest-ranked, the results still narrow down the area of concern and prompt further manual inspection that eventually reveals the erroneous subtraction of `NOZCROSS`.

The complete table of the top 10 suspicious statements is provided in Appendix A.

II. TCAS2

A. Analysis of Suspicious Statements

(a) *Faulty Statement Identification:* The fault in TCAS2 is in the `alt_sep_test()` function. The error occurs in the `else` clause where the value of `alt_sep` is set to `UPWARD_RA` instead of `UNRESOLVED`. The faulty line is:

```
alt_sep = UPWARD_RA;  
// Mutant: changed UNRESOLVED to UPWARD_RA
```

This is reported at Line 136 with an extremely high suspiciousness score of 37822500000.0, making it the top-ranked statement. Thus, `fl_dstar` has successfully identified the faulty statement as the most suspicious.

(b) *Impact on Debugging:* Since the faulty line is ranked highest, a developer would be immediately directed to the problematic assignment at Line 136. This direct indication would streamline the debugging process, allowing the developer to quickly examine the erroneous use of `UPWARD_RA` in place of `UNRESOLVED`. As a result, the tool effectively narrows down the search for the fault.

The complete table of the top 10 suspicious statements is provided in Appendix A.

III. TCAS3

A. Analysis of Suspicious Statements

(a) *Faulty Statement Identification:* In TCAS3, all statements have a suspiciousness score of 0.0 because no failing tests were recorded; every line shows 0 failures. As a result, `fl_dstar` does not identify any statement as particularly suspicious. This indicates that the coverage pattern is uniform across all tests—each statement is executed only in passing tests, making it impossible for the DStar metric to isolate a faulty line.

(b) *Impact on Debugging:* Since `fl_dstar` yields a suspiciousness of 0.0 for every statement, a developer would not receive any helpful ranking to pinpoint the fault. In this case, the lack of failing test cases (or the uniform execution in passing tests) forces the developer to rely on alternative debugging techniques, such as manual code inspection or different fault localization strategies, to locate the error.

The complete table of the top 10 suspicious statements is provided in Appendix A.

IV. TCAS4

A. Analysis of Suspicious Statements

(a) *Faulty Statement Identification:* In TCAS4 the most suspicious statements are at Lines 73, 74, 91, and 92, each with a suspiciousness score of 10.52. These lines evaluate conditions involving the result of `Own_Below_Threat()`. Although the actual fault is the change in the `NOZCROSS` constant (from 100 to 1000 in `Inhibit_Biased_Climb()`), the DStar analysis ranks these conditional statements higher. This is because these lines are executed almost exclusively by failing tests, creating a strong correlation between their execution and test failures, while the faulty constant's effect is distributed over multiple conditions.

(b) *Impact on Debugging:* A developer using these DStar results would initially focus on the highly ranked conditional statements (Lines 73, 74, 91, and 92). By examining these lines and tracing the decision logic, they would eventually scrutinize the function `Inhibit_Biased_Climb()`, where the `NOZCROSS` constant is used. This indirect guidance helps narrow the search area to the decision-making process, ultimately revealing that the constant is set incorrectly (1000 instead of 100). Thus, even though DStar does not directly flag the constant definition, it effectively directs attention to the critical code region.

The complete table of the top 10 suspicious statements is provided in Appendix A.

V. EFFECTIVENESS AND LIMITATIONS OF DSTAR FAULT LOCALIZATION

A. When DStar Performs Well

The DStar fault localization technique is particularly effective in scenarios where there is a strong correlation between test failures and specific lines of code. Some conditions where DStar is likely to perform well include:

1) *1. Faults That Directly Cause Test Failures:* DStar works well when the faulty statement is directly executed in most failing test cases and rarely in passing test cases. In such cases, the suspiciousness score will be high, allowing the faulty statement to be easily identified.

For example, in TCAS2, the faulty statement at Line 136 (`'alt_sep = UPWARD_RA;'`) was directly responsible for incorrect outputs and was executed in every failing test but never in passing tests. This resulted in an extremely high suspiciousness score, making it the top-ranked statement.

2) *2. Faults in Decision-Making Code:* Statements involved in critical control flow and decision-making logic (such as 'if' conditions and return values) are often ranked highly by DStar when they are executed predominantly in failing tests. This happens because the program's branching logic significantly affects its correctness.

For instance, in TCAS4, the top-ranked statements were inside the `'Own_Below_Threat()'` and `'Own_Above_Threat()'` decision structures. These conditions were more frequently executed in failing tests than in passing tests, leading to high suspiciousness scores.

3) *3. Programs with Clear Test Failure Patterns:* DStar is most effective when the test suite provides a strong distinction between passing and failing test cases. If failing test cases consistently reach a certain part of the program while passing ones do not, those lines will have high suspiciousness scores.

For instance, in TCAS1, while the actual fault (in `'Inhibit_Biased_Climb()'`) was not the highest-ranked, the tool still highlighted control flow statements that were highly correlated with failures, allowing a developer to trace back to the root cause.

B. When DStar Performs Poorly

Despite its strengths, DStar has limitations in certain scenarios, particularly when failing and passing test cases exhibit

similar execution patterns or when faults are more complex. Some cases where DStar struggles include:

1) *1. Faults That Are Indirectly Responsible for Failures:* If a fault exists in a line of code but its effects propagate through multiple function calls before causing a failure, DStar may fail to assign it a high suspiciousness score. This happens because the faulty line is executed in both passing and failing tests, reducing its correlation with failures.

For example, in TCAS1, the actual faulty statement (`'Inhibit_Biased_Climb()'`) was not the highest-ranked because it influenced program behavior indirectly. Instead, statements that used its return value in conditional decisions were ranked higher due to stronger execution correlation with failures.

2) *2. When There Are No Failing Test Cases:* DStar relies entirely on coverage differences between failing and passing test cases. If all tests pass, DStar assigns a suspiciousness score of zero to every line, making it impossible to identify faults.

This occurred in TCAS3, where no failing test cases were recorded. As a result, all statements had a suspiciousness score of 0.0, and the tool provided no meaningful guidance on fault localization.

3) *3. Faults in Initialization Code or Constant Definitions:* Faults in constants, global variables, or initialization routines can be difficult for DStar to detect because these statements execute the same way in every test case, making them indistinguishable based on coverage differences.

For instance, in TCAS4, the actual fault was the modification of the constant `'NOZCROSS'`. Since this constant was used across multiple functions, no single statement exhibited a clear execution correlation with failures, making it harder for DStar to directly highlight the root cause.

4) *4. Programs with High Redundant Execution:* If a program contains a loop or redundant function calls that are executed frequently in both passing and failing tests, DStar may rank those lines lower than expected. This is because the denominator of the DStar formula increases, reducing the suspiciousness score of frequently executed statements.

For example, if a faulty computation is performed inside a loop that executes in all test cases, the line might appear unremarkable despite causing failures. DStar struggles in such cases because it does not account for execution frequency beyond the raw failed/passed ratio.

C. Conclusion

DStar is a powerful statistical fault localization technique that performs well when:

- The faulty statement is executed almost exclusively in failing test cases.
- The fault is in decision-making logic, leading to distinct test behaviors.
- The test suite provides strong differentiation between passing and failing tests.

However, it performs poorly when:

- The fault propagates through multiple functions before affecting output.

- No failing test cases exist, leading to a complete failure of the ranking.
- The fault is in initialization code or a constant that does not directly cause execution divergence.
- The faulty statement is inside a frequently executed loop or function, reducing its suspiciousness score.

This homework and report are completed with the help of resources provided along with the homework. This report also makes use of o3-mini to refine the content.

APPENDIX

Line	Statement	#Failed	#Passed	Suspiciousness
134	alt_sep = DOWNWARD_RA;	27	125	5.52
77	result = Own_Above_Threat()	34	437	2.65
95	... result = !(Own_Above_Threat())	34	437	2.65
105	... bool Own_Above_Threat()	34	595	1.94
107	return (Other_Tracked_Alt < Own_Tracked_Alt);	34	595	1.94
133	else if (need_downward_RA)	34	713	1.62
59	int Inhibit_Biased_Climb()	34	850	1.36
61	return (Climb_Inhibit ? Up_Separation + NOZCROSS ...	34	850	1.36
64	bool Non_Crossing_Biased_Climb()	34	850	1.36
70	upward_preferred = Inhibit_Biased_Climb() ...	34	850	1.36

TABLE I

TOP 10 SUSPICIOUS STATEMENTS IN TCAS1. (...) INDICATE TRUNCATED CODE TO FIT THE TABLE.

Line	Statement	#Failed	#Passed	Suspiciousness
136	alt_sep = UPWARD_RA; // Mutant: changed UNRES...	615	0	378225000000.0
133	else if (need_downward_RA)	615	125	3025.8
59	int Inhibit_Biased_Climb()	615	269	1406.04
61	return (Climb_Inhibit ? Up_Separation + NOZCRO...	615	269	1406.04
64	bool Non_Crossing_Biased_Climb()	615	269	1406.04
70	upward_preferred = Inhibit_Biased_Climb() ; Do...	615	269	1406.04
71	if (upward_preferred)	615	269	1406.04
79	return result;	615	269	1406.04
82	bool Non_Crossing_Biased_Descend()	615	269	1406.04
88	upward_preferred = Inhibit_Biased_Climb() ; Do...	615	269	1406.04

TABLE II

TOP 10 SUSPICIOUS STATEMENTS IN TCAS2. (...) INDICATE TRUNCATED CODE TO FIT THE TABLE.

Line	Statement	#Failed	#Passed	Suspiciousness
46	void initialize()	0	1576	0.0
48	Positive_RA_Alt_Thresh[0] = 400;	0	1576	0.0
49	Positive_RA_Alt_Thresh[1] = 500;	0	1576	0.0
50	Positive_RA_Alt_Thresh[2] = 640;	0	1576	0.0
51	Positive_RA_Alt_Thresh[3] = 740;	0	1576	0.0
52	{ }	0	1576	0.0
54	int ALIM ()	0	562	0.0
56	return Positive_RA_Alt_Thresh[Alt_Layer_Value];	0	562	0.0
59	int Inhibit_Biased_Climb()	0	884	0.0
61	return (Climb_Inhibit ? Up_Separation + NOZCROSS ...	0	884	0.0

TABLE III

TOP 10 SUSPICIOUS STATEMENTS IN TCAS3. (...) INDICATE TRUNCATED CODE TO FIT THE TABLE.

Line	Statement	#Failed	#Passed	Suspiciousness
73	result = !(Own_Below_Threat()) ...	79	593	10.52
74	{ }	79	593	10.52
91	result = Own_Below_Threat() ^ (Cur_Vertical_Sep ...	79	593	10.52
92	{ }	79	593	10.52
100	bool Own_Below_Threat()	79	673	9.27
102	return (Own_Tracked_Alt < Other_Tracked_Alt);	79	673	9.27
59	int Inhibit_Biased_Climb()	79	805	7.75
61	return (Climb_Inhibit ? Up_Separation + NOZCRO...	79	805	7.75
64	bool Non_Crossing_Biased_Climb()	79	805	7.75
70	upward_preferred = Inhibit_Biased_Climb() > ...	79	805	7.75

TABLE IV

TOP 10 SUSPICIOUS STATEMENTS IN TCAS4. (...) INDICATE TRUNCATED CODE TO FIT THE TABLE.